SecureVault

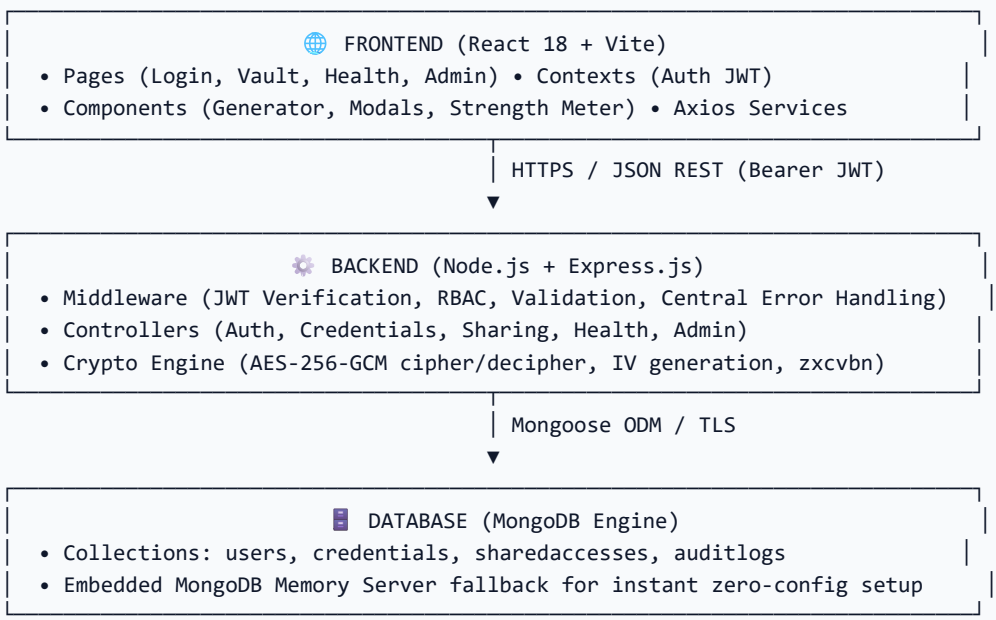
Enterprise Organization Credential Management Platform (MERN Stack)

Comprehensive Architecture, Modules & Files Reference Manual

AES-256-GCM ENCRYPTION | ROLE-BASED ACCESS CONTROL | ZERO-KNOWLEDGE ARCHITECTURE | TAMPER-EVIDENT AUDIT LOGGING

1. System Architecture & High-Level Module Overview

SecureVault isolates responsibility across 6 distinct project directories. Every request enforces authenticated sessions, input validation, cryptographic confidentiality, and audit tracking.



> **Zero-Knowledge Security Invariant:** System administrators possess governance authority (suspending users, setting access tiers, inspecting audit logs), but the cryptographic architecture prevents administrators or databases from decrypting passwords of accounts they do not own.

2. Frontend Module (`frontend/`)

The presentation tier built with React 18, Vite, and modern CSS glassmorphism components.

2.1 Core & Infrastructure Files

File Path	Primary Job & Responsibilities
`frontend/index.html`	Single Page Application entry shell. Mounts root DOM container `#root`, defines viewport responsiveness, and configures document title.
`frontend/vite.config.js`	Vite build configuration. Integrates `@vitejs/plugin-react` and configures reverse-proxy from `/api` to `http://localhost:5000` for seamless local development.
`frontend/package.json`	Frontend dependencies manifest (React 18, React Router v6, Axios, Lucide React, zxcvbn) and scripts (`dev` , `build` , `preview`).
`frontend/src/main.jsx`	Application bootstrap entry point. Initializes React root and renders `<App />` wrapped in strict rendering mode.
`frontend/src/App.jsx`	Top-level routing orchestrator. Configures client routes, layout frames (Navbar, Sidebar), and protects secure endpoints via `<ProtectedRoute>` .
`frontend/src/index.css`	Comprehensive design system. Implements color tokens, glassmorphism card surfaces, custom form controls, animations, and responsive utilities.

2.2 Application State & API Services

File Path	Primary Job & Responsibilities
`frontend/src/context/AuthContext.jsx`	Central authentication state. Manages JWT lifecycle in `localStorage` , decodes user credentials, and provides `login()` , `register()` , and `logout()` hooks.
`frontend/src/services/api.js`	Centralized Axios HTTP instance. Automatically intercepts all outgoing requests and injects the `Authorization: Bearer <token>` header.
`frontend/src/services/authService.js`	Executes authentication endpoints: login requests, user registration, and active session profile resolution (`/api/auth/me`).
`frontend/src/services/credentialService.js`	Executes personal vault actions: listing credentials, adding encrypted items, triggering decryption reveal, modifying entries, and managing colleagues sharing.
`frontend/src/services/healthService.js`	Retrieves personal password hygiene metrics: weak passwords count, reused credentials count, and passwords older than 90 days.
`frontend/src/services/adminService.js`	Administrative client services: fetching organisation KPI metrics, toggling employee activation states, promoting user roles, and streaming compliance audit logs.

2.3 Components & UI Widgets

File Path	Primary Job & Responsibilities
`.../common/TopNavbar.jsx`	Global navigation bar displaying brand header, active session indicators, authenticated user profile chip, and one-click Logout.

File Path	Primary Job & Responsibilities
`.../common/Sidebar.jsx`	Responsive left navigation. Renders employee links (Vault, Password Health) and privileged administrator links (Dashboard, Users, Health, Audit Trail).
`.../common/ProtectedRoute.jsx`	Route security guard. Validates token presence and required role privileges; redirects unauthenticated visitors to `/login`.
`.../common/LiveStrengthMeter.jsx`	Real-time password entropy meter powered by `zxcvbn`. Calculates scores (0–4) with interactive color-coded visual bars and crack-time estimation.
`.../credentials/CredentialDetailModal.jsx`	Detail modal with on-demand zero-knowledge decryption. Clicking **Reveal** fetches decrypted password from API with 1-click clipboard copy.
`.../credentials/CredentialFormModal.jsx`	Add/Edit credential dialog. Collects service title, username, password, and URL. Features live password generator integration.
`.../credentials/GeneratorWidget.jsx`	High-entropy password generator. Adjustable length slider (8–64 chars) with toggles for uppercase, lowercase, numbers, and special symbols.
`.../credentials/ShareModal.jsx`	Secure credential delegation dialog. Lists colleagues with search, assigns `view` or `edit` permissions, and manages active access revocations.

2.4 Application Pages

File Path	Primary Job & Responsibilities
`frontend/src/pages/LoginPage.jsx`	Authentication view with demo account quick-fill selectors for fast evaluation.
`frontend/src/pages/RegisterPage.jsx`	Employee registration page enforcing master password complexity via live feedback.
`frontend/src/pages/CredentialsPage.jsx`	Primary personal vault dashboard. Displays stored and shared credentials with filtering, search, reveal, edit, and sharing triggers.
`frontend/src/pages/HealthPage.jsx`	Personal password hygiene diagnostics displaying overall score and flags for weak, reused, or expired credentials.
`frontend/src/pages/AdminDashboardPage.jsx`	Executive summary view displaying total users, credentials managed, organisation health index, and recent audit activity.
`frontend/src/pages/AdminUsersPage.jsx`	Governance dashboard allowing administrators to activate/deactivate accounts or promote/demote employee roles.
`frontend/src/pages/AdminHealthPage.jsx`	Enterprise health audit view reporting vulnerability counts per employee without ever exposing plaintext secrets.
`frontend/src/pages/AdminAuditPage.jsx`	Compliance audit trail showing tamper-evident log records with timestamp, actor email, action type, credential title, and IP address.

3. Backend Module (`backend/`)

The application server layer built on Node.js and Express.js providing authentication, access control, and cryptography.

File Path	Primary Job & Responsibilities
`backend/server.js`	Server entry point. Mounts CORS, express body parsers, routes, error handler, and serves client static build in production. Automatically loads `config/.env`.
`backend/package.json`	Backend dependencies (`express`, `mongoose`, `bcryptjs`, `jsonwebtoken`, `mongodb-memory-server`, `zxcvbn`) and run scripts.
`backend/routes/authRoutes.js`	Declares authentication endpoints: `POST /register`, `POST /login`, and `GET /me`.
`backend/routes/credentialRoutes.js`	Declares credential lifecycle endpoints: list, create, reveal/decrypt, update, delete, and delegate share permissions.
`backend/routes/healthRoutes.js`	Declares password health diagnostic endpoint: `GET /api/health/me`.
`backend/routes/adminRoutes.js`	Declares administrative governance endpoints protected by `requireAdmin`: stats, user status toggling, role assignment, and audit logs.
`backend/controllers/authController.js`	Authenticates users, generates signed JWT tokens, enforces bcrypt hashing, and logs login security events.
`backend/controllers/credentialController.js`	Performs AES-256-GCM encryption on save, handles zero-knowledge on-demand decryption upon reveal, checks user ownership, and records audit logs.
`backend/controllers/shareController.js`	Manages shared credentials: creates/updates `SharedAccess` documents, enforces recipient checks, and handles revocation.
`backend/controllers/healthController.js`	Performs password health checks: computes strength scores, checks password age (>90 days), and detects cross-credential password reuse via in-memory hashing.
`backend/controllers/adminController.js`	Computes organization-wide security metrics, updates user roles and status, and streams filterable compliance logs.
`backend/middleware/auth.js`	JWT verification middleware (`authenticateToken`). Validates bearer token, confirms user active status in database, and attaches `req.user`.
`backend/middleware/rbac.js`	Role-Based Access Control guard (`requireAdmin`). Rejects non-admin users with an HTTP 403 Forbidden response.
`backend/middleware/validate.js`	Input validation suite using `express-validator` for emails, strong passwords, URLs, and permissions.
`backend/middleware/errorHandler.js`	Central error handler catching unhandled exceptions and returning sanitized, standardized JSON error responses.
`backend/utils/crypto.js`	Cryptographic utility implementing AES-256-GCM authenticated cipher/decipher with random 16-byte IV and auth tag validation, plus cryptographic password generation.
`backend/utils/passwordHealth.js`	Entropy calculation engine wrapping `zxcvbn` to return password scores (0 to 4) and remediation feedback.

File Path	Primary Job & Responsibilities
`backend/utils/auditLogger.js`	Tamper-evident audit logger (`logAuditEvent`). Asynchronously writes audit records to MongoDB with actor ID, action type, IP address, and timestamp.

4. Database Module (`database/`)

MongoDB Mongoose schemas, auto-reconnecting database engine, in-memory fallback, and seeding scripts.

File Path	Primary Job & Responsibilities
`database/config/db.js`	Database connection manager. Tries connecting to `MONGO_URI` ; if local MongoDB is unavailable, automatically spawns an embedded **MongoMemoryServer** so the project runs out-of-the-box.
`database/models/User.js`	User Mongoose schema (`name` , unique `email` , `masterPasswordHash` , `role` , `isActive` , `createdAt`). Contains bcrypt hashing helper methods.
`database/models/Credential.js`	Credential schema (`ownerId` , `title` , `username` , `url` , `encryptedPassword` , `iv` , `authTag` , `strengthScore` , `lastUpdated`). Plaintext passwords are never stored.
`database/models/SharedAccess.js`	SharedAccess schema (`credentialId` , `sharedWith` , `sharedBy` , `permission: 'view'|'edit'` , `createdAt`) for granular employee sharing.
`database/models/AuditLog.js`	AuditLog schema (`userId` , `action` , `credentialId` , `credentialTitle` , `details` , `ipAddress` , `timestamp`) indexed for fast compliance queries.
`database/seed.js`	Self-contained database seeder. Populates sample accounts (Admin: Meera, Employees: Aditya, Rahul, Sneha, Kiran), encrypted credentials, shares, and audit logs.
`database/sample_data.json`	Static JSON fixtures export for reference and external testing.
`database/README.md`	Dedicated documentation explaining database schemas, encryption fields, and seeding instructions.

5. Configuration, Utilities & Root Files

File Path	Primary Job & Responsibilities
`config/.env`	Active local configuration file defining `PORT` , `MONGO_URI` , `JWT_SECRET` , `ENCRYPTION_KEY` , and `CLIENT_URL` . Excluded from Git.
`config/.env.example`	Environment variables template committed to version control with guidance on generating secure encryption keys.
`others/postman_collection.json`	Complete Postman test collection (v2.1) covering all REST endpoints for API validation.
`others/generate_diagrams.py`	Python Pillow script that generates high-resolution architectural, class, and sequence diagram images in `docs/diagrams/` .

File Path	Primary Job & Responsibilities
`others/update_report.py`	Python utility that embeds updated diagrams and screenshots into the formal Word report document.
`docs/`	Houses project presentations (Review 0 & 1), full-resolution UI screenshots, generated diagrams, and project reports. Kept intact as requested.
`package.json`	Root orchestrator with concurrent scripts (`npm run dev` , `npm run server` , `npm run client` , `npm run seed` , `npm run build`).
`SecureVault.code-workspace`	VS Code workspace definition organizing the repository into Frontend, Backend, Database, and Docs folders.
`.gitignore`	Git exclusion rules for dependencies (`node_modules/`), build outputs (`dist/`), and secrets (`config/.env`).
`render.yaml`	Infrastructure configuration for 1-click cloud deployment on Render.
`README.md`	Comprehensive project guide with setup instructions, security principles, and quick-start commands.
`MODULES.md`	Markdown version of this complete modules and files reference guide.

6. End-to-End Request & Security Lifecycle

How Data Flows Through The System:

1. **UI Action:** User enters a password in `CredentialFormModal.jsx` ; `LiveStrengthMeter.jsx` computes entropy score (0–4) locally.
2. **API Transmission:** `credentialService.js` sends a `POST /api/credentials` request with the Bearer JWT token in the `Authorization` header.
3. **Auth Verification:** `auth.js` middleware decodes the JWT, validates user active state against the `User` model, and attaches `req.user` .
4. **AES-256-GCM Encryption:** `credentialController.js` passes the plaintext secret to `crypto.js` , which generates a fresh 16-byte random IV, creates a Galois Counter Mode cipher, and extracts a 16-byte authentication tag.
5. **Database Persistence:** Only ciphertext, IV, and auth tag are written to MongoDB via `Credential.js` . Plaintext is discarded from memory.
6. **Audit Logging:** `auditLogger.js` writes an immutable record to `AuditLog.js` recording user identity, action (`create`), credential title, client IP, and timestamp.
7. **On-Demand Decryption (Reveal):** When an authorized user requests reveal, `credentialController.js` verifies ownership or `SharedAccess.js` permissions, decrypts the ciphertext using the stored IV and auth tag, and streams the decrypted password back to the client.